

Problem Statement and Real User Need

Construction accounting is a specialized field where cost accountants calculate and estimate the costs associated with various aspects of construction projects. This process is traditionally long and labor intensive, as it involves accounting for multiple types of costs according to detailed, predefined tables. In Georgia, as in many post-Soviet countries, these methods have remained largely unchanged since the Soviet era. For example, cost accountants in Georgia still rely on thousands of tables listing materials, labor, and other cost metrics. They manually search these tables to compile construction accounting files that comply with legal requirements.

This workflow is not only complex and time-consuming but also prone to human error. To address this, we propose an automated solution that streamlines the construction accounting process. Our software will allow cost accountants to focus on decision-making rather than manual data entry, accelerating the workflow and improving accuracy. Additionally, business owners who rely on cost accountants will benefit from faster, more reliable reporting and cost estimates.

Our team has been developing this idea into a viable business solution for some time. We have consulted with professional accountants and completed construction accounting courses to ensure we understand the field thoroughly. Discussions with leading cost accountants in Georgia revealed that the process has remained largely unchanged for decades. One expert even attempted to develop software to modernize the workflow but was unable to do so due to technical limitations. Our team's combination of domain knowledge and technical capability positions us well to address this longstanding challenge effectively.

Proposed Solution and AI-Powered Differentiator

Our proposed solution focuses on automating the construction accounting workflow. The core of this solution involves transforming thousands of cost reference tables into a structured and queryable database. We are considering technologies such as vector databases optimized for Retrieval-Augmented Generation (RAG), as well as document-oriented systems like MongoDB, which support complex nested data structures. This transformation will enable efficient data retrieval and automated cost calculations.

A key technical challenge lies in the conversion of the existing tables, as they vary significantly in format and structure. Traditional scripting approaches alone are insufficient for handling this variability at scale. To address this, we propose developing an AI-driven agent capable of generating tailored data-conversion scripts for each table.

This approach allows us to systematically standardize and migrate legacy data into a unified database architecture.

On the user side, the platform will allow clients to upload project specific documents containing construction details. Based on this input, the system will automatically generate compliant construction accounting files. This component will primarily rely on robust backend engineering to ensure accuracy, consistency, and regulatory compliance.

Artificial intelligence will play a particularly critical role in the platform's conversational interface. We will implement an advanced chatbot that enables both cost accountants and business owners to ask detailed, context-specific questions also using speech to text technologies. The system will provide precise, case-based responses, effectively functioning as a virtual professional accountant capable of explaining concepts clearly and supporting informed decision-making.

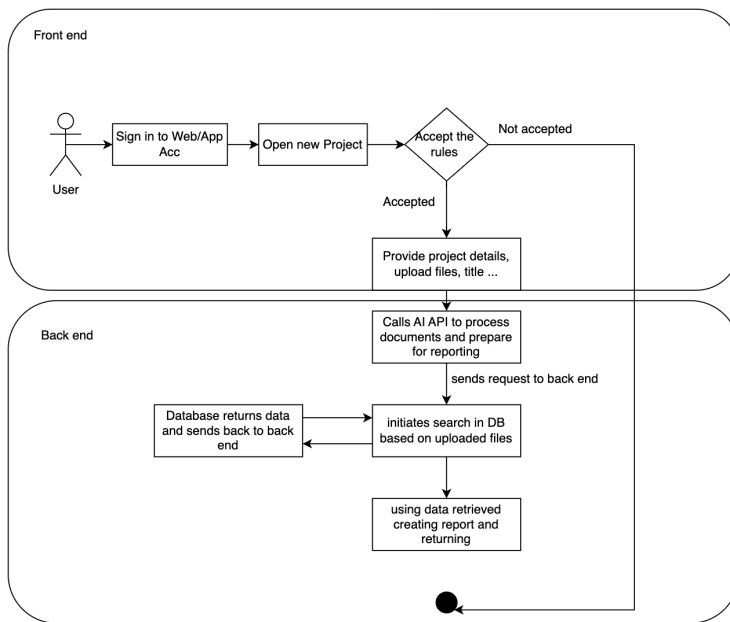
Without AI implemented our software would lack very important thing which is well trained chatbot, we can create one without AI as we can transform tables into databases manually but in both cases we would have lower quality. Manual work on table conversions would take most of our time that can be spent on design and logic, while simple chatbot would lack depth and understanding of topics.

Technical Architecture (with diagram)

The system is organized into two primary user flows, both sharing the same authentication gateway and a common backend infrastructure layer. The frontend is a web/mobile application; the backend handles AI orchestration, database queries, and file generation.

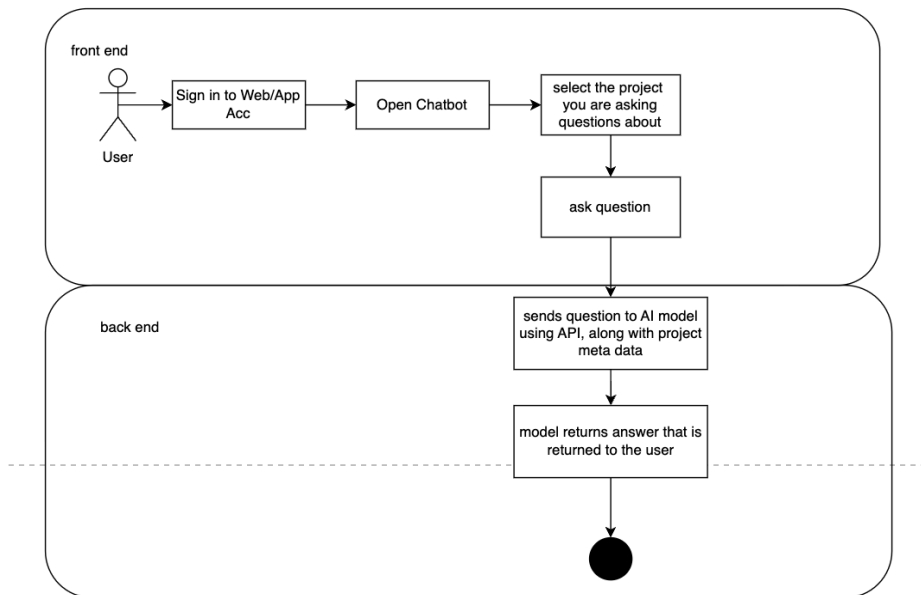
Flow 1 — Project creation & report generation

The user authenticates, accepts a data processing agreement (which serves as a lightweight digital contract), and submits project metadata along with source documents. The frontend passes these files to the backend, which calls an AI API to extract and structure the raw content. The backend then queries the database for reference data, assembles all retrieved information, and generates a downloadable report file that is returned to the client.



Flow 2 — Chatbot Q&A on a project

Once a project exists, the user can open the chatbot interface, select which project they want to query, and ask free-form questions. The backend receives the question alongside the project's stored metadata and passes both to the AI model via API. The model's answer is streamed or returned directly to the user — no database write occurs in this path.



Authentication is shared across both flows. A single sign-in gateway covers both the project creation surface and the chatbot, with session tokens reused across routes.

Contract acceptance in Flow 1 acts as a prerequisite gate - the user cannot proceed to file upload until they agree to the data processing terms. This is enforced at the backend route level, not just the UI.

AI API usage differs between flows: in Flow 1 the AI does heavy lifting - parsing uploaded documents, structuring data, and feeding it into the report builder. In Flow 2 the AI receives a lightweight context package (the question plus stored project metadata) and generates a direct answer, with no new writes to the database.

The database is only involved in Flow 1, where it stores and retrieves reference data needed to populate the report. Flow 2 draws on pre-processed project metadata already cached at submission time.

Technology Stack

This is not final yet, we will decide on specific technologies as software is developed

Layer	Technology	Why this choice
Frontend framework	React (Next.js)	Server-side rendering improves performance; ideal for dashboard and chatbot UI
UI library / styling	Tailwind CSS	Fast, consistent styling with minimal overhead
Backend language	Node.js (TypeScript)	Strong typing and excellent for scalable API systems
Backend framework	NestJS	Structured, modular architecture suitable for complex workflows
Database (primary)	MongoDB	Handles highly variable construction table schemas
Vector database	Pinecone	Fast similarity search for RAG-based chatbot responses
AI model — document parsing	GPT-4o	Strong at extracting structured data from PDFs and mixed-format

		documents
AI model — chatbot	GPT-4.1	High reasoning accuracy for professional Q&A and explanations
AI model — embeddings	text-embedding-3-large	High-quality embeddings for semantic search in RAG pipeline
AI model — speech-to-text	Whisper-1	Accurate transcription for voice-based queries
AI processing	RAG pipeline + custom AI agents	Ensures accurate, context-aware responses grounded in real data
File storage	AWS S3	Scalable and secure storage for uploaded documents
File processing	pdf-parse + Tesseract OCR	Handles both digital and scanned documents
Report generation	PDFKit / ExcelJS	Generates compliant construction accounting reports
Authentication	JWT + OAuth (optional)	Secure and scalable user management

Hosting — frontend	Vercel	Optimized for Next.js with easy deployment
Hosting — backend	AWS (EC2 / Lambda)	Scalable infrastructure for AI-heavy workloads
Containerization	Docker	Ensures consistent environments across development and deployment
CI/CD	GitHub Actions	Automates testing and deployment workflows
Caching / queue	Redis + BullMQ	Handles heavy AI jobs and improves performance
Monitoring	Sentry	Real-time error tracking and debugging
Version control	GitHub	Team collaboration and version management

Risk and Failure Mode Analysis

Developing an AI-powered construction accounting platform introduces several technical, operational, and market risks. Identifying these early allows us to design mitigation strategies and improve system reliability.

1. Data Conversion Risks

Legacy construction tables vary in format, structure, and completeness. AI-generated scripts may misinterpret certain formats, leading to incorrect database entries.

Failure Mode:

- Incorrect cost mappings
- Missing data fields
- Inconsistent schema across tables

Mitigation:

- Human-in-the-loop validation during early stages
- Automated validation rules (e.g., range checks, required fields)
- Version-controlled data pipelines to track changes and roll back errors

2. AI Output Accuracy (Critical Risk)

The AI system (both document parser and chatbot) may generate incorrect or misleading outputs.

Failure Mode:

- Incorrect cost estimates in reports
- Misleading answers in chatbot responses
- Hallucinated data not grounded in database

Mitigation:

- Retrieval-Augmented Generation (RAG) to anchor responses in verified data
- Confidence scoring and highlighting uncertain outputs
- Restrict chatbot to cite sources (tables, regulations)
- Continuous fine-tuning using real accountant feedback

3. Regulatory and Compliance Risk

Generated reports may fail to meet Georgian construction accounting standards.

Failure Mode:

- Reports rejected by authorities
- Legal liability for incorrect filings

Mitigation:

- Close collaboration with certified cost accountants
- Rule-based validation layer before report generation
- Regular updates aligned with regulatory changes

Team Roles and Week-by-Week Plan

Guga — Backend & System Architecture

- Design overall system architecture
- Implement backend services and APIs
- Manage database design (MongoDB / vector DB)
- Ensure system scalability and performance

Nikoloz — AI & Data Engineering

- Develop AI agent for table conversion
- Implement RAG pipeline and chatbot logic
- Handle document parsing and structuring
- Work on model evaluation and accuracy improvements

Anastasia — Product, UX & Business

- Define user flows and interface requirements
- Coordinate with accountants and domain experts
- Ensure regulatory compliance in outputs
- Manage documentation, testing, and user feedback

Week 1 — Research & Planning

- Finalize requirements with accountants
- Define system architecture
- Assign responsibilities clearly

Week 2 — Data Analysis

- Collect and analyze construction tables
- Identify patterns and variations
- Design initial database schema

Week 3 — Prototype Database

- Build initial MongoDB structure
- Manually input sample tables
- Test query performance

Week 4 — AI Conversion Prototype

- Develop first version of AI table-to-JSON converter
- Test on small dataset
- Evaluate accuracy

Week 5 — Backend Development

- Build authentication system
- Implement project creation API
- Set up file upload pipeline

Week 6 — Document Processing

- Integrate AI for document parsing
- Structure extracted data
- Connect to database queries

Week 7 — Report Generation

- Build report generation engine
- Ensure compliance with accounting formats
- Export downloadable files

Week 8 — Chatbot Development

- Implement RAG-based chatbot
- Connect chatbot to project metadata
- Test Q&A functionality

Week 9 — Testing & Validation

- End-to-end system testing
- Validate outputs with accountants
- Fix bugs and improve accuracy

Week 10 — Finalization & Demo

- Polish UI/UX
- Optimize performance
- Prepare presentation and demo

IRB-Light Checklist

1. Does your application collect images of real people?

No.

2. Does your application process photographs of faces?

No.

3. Does your application handle sensitive documents (medical, legal, financial, government IDs)?

Yes. The application will process financial documents as well as confidential construction related information, which may in some cases be classified as sensitive business data.

4. Does your application store user-uploaded data? If yes, for how long and where?

Yes. Users will upload documents containing construction project details in order to generate construction accounting files. These files may be stored either in a database (to support structured processing and retrieval) or as secure file storage, depending on the final system architecture. Data retention will be project-based: user data will be stored only for the duration of the project and will be deleted once the project is completed and the user finalizes the accounting outputs.

5. Do users need to understand what happens to their data before using the application?

Yes. Since the application processes sensitive business and financial information, it is essential that users clearly understand how their data is collected, used, and stored before proceeding.

6. If any of the above are “yes”: describe your consent flow. How does a user know what data is collected and how it is used?

Users will be presented with a clear and concise data usage and privacy notice at the beginning of each new project. This notice will explain what data is collected, how it will

be used, how long it will be stored, and the measures taken to protect it. Users must explicitly provide consent before proceeding to the operational phase of the application.